CHECK Documentation

Cultural Heritage Experiment Configuration Kit

Manuele Veggi
CNR Institute of Heritage Science, Florence; Sapienza University of Rome
manueleveggi@cnr.it

Version 1.0.0 — June 2, 2026

Check
Cultural Heritage Experiments
Configuration Kit

Overview

CHECK is a web application built on the ATON framework, designed to support curators, cultural heritage professionals, and behavioural scientists in configuring and conducting controlled experiments in immersive 3D environments, without requiring programming skills.

This document provides both a **user guide** (Part A) for researchers and practitioners, and a **technical reference** (Part B) for developers who wish to extend or integrate the system. CHECK was developed at CNR — Institute of Heritage Science and represents the second design iteration of the EXPERIMENT PLANNER prototype.

Website: https://cnr-ispc-dhilab-fi.github.io/checK/

GitHub: https://github.com/cnr-ispc-dhilab-fi/checK/

Contents

1 Introduction. Context and Motivation 4

I User Guide 5

2 System Requirements & Installation 5

2.1 Requirements 5

2.2 Installation 5

3 Roles and Interfaces Overview 6

4 Configuring a New Project 6

4.1 Protocol Definition 7

4.2 Project Overview Page 9

5 Running an Experimental Session 9

5.1 Subject's Interface 10

5.2 Tester's Interface 10

6 Records Visualisation and Data Export 11

II Technical Reference 13

7 Architecture and Codebase Structure 13

8 Style Sheet and Visual Identity 13

9 Web Application Components 15

9.1 Setup and Storage Architecture 15

9.1.1 Aton App Storage 15

9.1.2 Server-Side Filesystem 15

9.2 New Project Creation 16

9.2.1 Project Initialisation 16

9.2.2 Project Form Wizard 17

9.2.3 GLB Thumbnail Generation 17

9.2.4 Protocol Setup 17

9.3 Experimental Session: Architecture and Runtime 18
9.3.1 Session Initialisation 19
9.3.2 Storage Snapshot 19
9.3.3 Tester Dashboard 19
9.3.4 Multimedia Delivery 20
9.3.5 Scene and Multi-User Layer 20
9.3.6 Phase Navigation and Data Capture 21
9.4 Data Visualisation and Bulk Export 22
9.4.1 Session Data Visualisation 22
9.4.2 Bulk CSV Export 23

10 Future Work **23**

1 Introduction. Context and Motivation

Immersive environments are increasingly transforming access to cultural heritage, opening new possibilities for the systematic analysis of visitor experience. Studies in cultural heritage and behavioural psychology share deep intersections: over the past decades, scholars in visitor studies have highlighted how cultural heritage spaces can be a privileged setting to study human behaviour. By the same token, psychologists and neuroscientists have long regarded these environments as rich in stimuli and socio-cultural significance, which further influences subjects' cognitive and emotional responses.

Despite the growing use of Virtual Reality (VR) in this domain, a persistent gap remains: the available tools for conducting controlled experiments in immersive 3D environments either require advanced programming skills or lack the flexibility needed by practitioners in cultural heritage and the social sciences. Frameworks such as Psychtoolbox [1, 7] and PsychoPy [10] support VR but demand expertise in Python or MATLAB; game engine add-ons such as UXF [2] still require both programming and game development skills. More accessible graphical tools, such as WorldViz's SightLab VR Pro [12], are instead proprietary, platform-limited, and not designed for the rendering of complex heritage environments.

In 2024, a *minimum viable product* (MVP) of an Experiment Planner was presented at the Eurographics GCH Workshop [8]. Developed at DHILab – CNR ISPC,[1] it consisted of a web application built on Aton, a liquid open-source framework for immersive, collaborative web applications in cultural heritage [4]. This first prototype was validated in a real experimental scenario on navigational styles, using the digital twin of a temporary exhibition. Although the pilot study confirmed the feasibility of the approach, a formative evaluation with behavioural scientists identified usability issues: the admin panel was not perceived as sufficiently intuitive, users experienced cognitive overload, and several manual procedures were required for session setup.

checK (Cultural Heritage Experiments Configuration Kit) is the second design iteration of this tool, developed in response to those findings. It is a modular web application built on Aton, designed to guide curators, cultural heritage professionals and behavioural scientists through the full cycle of an experimental session. It supports researchers from task configuration and stimulus delivery to real-time observation and standardised data collection, without requiring any programming skills. This document provides both a user guide (Part A) and a technical reference for developers (Part B). checK is conceived as part of the pilot of Task 7.9 of the H2IOSC project and of a broader design ecosystem that also includes the Italian national project CHANGES, and the Horizon Europe projects PERCEIVE and COLOURS.

[1] https://www.ispc.cnr.it/

I. User Guide

2 System Requirements & Installation

2.1 Requirements

- Modern web browser (Chrome, Firefox) and internet connection
- Main ATON framework instance (v3.0.2)[2]
- Node.js $\geq$ v24.14.0

2.2 Installation

0. **Meet the requirements.** Clone the main ATON repository from GitHub and install needed packages with Node Package Manager:

```
git clone https://github.com/phoenixbf/aton.git
npm install
```

Clone dependencies from Interlumo:

- In the folder `config/`, clone Kapto and Merkhet flares:

```
git clone https://github.com/phoenixbf/kapto.git
git clone https://github.com/phoenixbf/merkhet-plugin.git
```

- In the folder `wapps/`, clone the Merkhet web-app:

```
git clone https://github.com/phoenixbf/merkhet-app
```

- Inside `wapps/merkhet/`, create `config.json` and set the Kapto Hub endpoint: `{"capturehub":"https://interlumo.ispc.cnr.it/kapto/"}`

1. **Set-up Users.** In the folder `data/`, create the required directories:

```
mkdir -p data/collections/check-user
mkdir -p data/scenes/check-user
```

2. **Download web-app and server.** Clone the following CHECK repositories:
   - In the folder `wapps/`, clone:

```
git clone https://github.com/cnr-ispc-dhilab-fi/checK.git
```

   - In the folder `config/`, clone:

```
git clone https://github.com/cnr-ispc-dhilab-fi/check-flare.git
```

3. Deploy the server and the application:

```
npm run deploy-pm2
```

4. Open your browser at `http://localhost:{port}/a/checK`.

[2] The framework can be cloned from its reference GitHub repository: https://github.com/phoenixbf/aton.

The directories mentioned in this tutorial are sub-folders of the ATON main instance.

3 Roles and Interfaces Overview

The application is built to meet the needs of two different roles:

- *tester*, i.e. the researcher or practitioners authoring and conducting the experiment;
- tested subject (or, simply, *subject*), i.e. the participant to the experiment whose behaviour is monitored and studied.

The first touchpoint of the application is tester's main dashboard (Fig. 1), where users can access previously created projects or create new ones. Conversely, subject can access the experiment from an onboarding form (see Sect. 5). This guide presents a sample tutorial, starting from the definition of a new project to the run of experimental sessions and the analysis of the collected data.

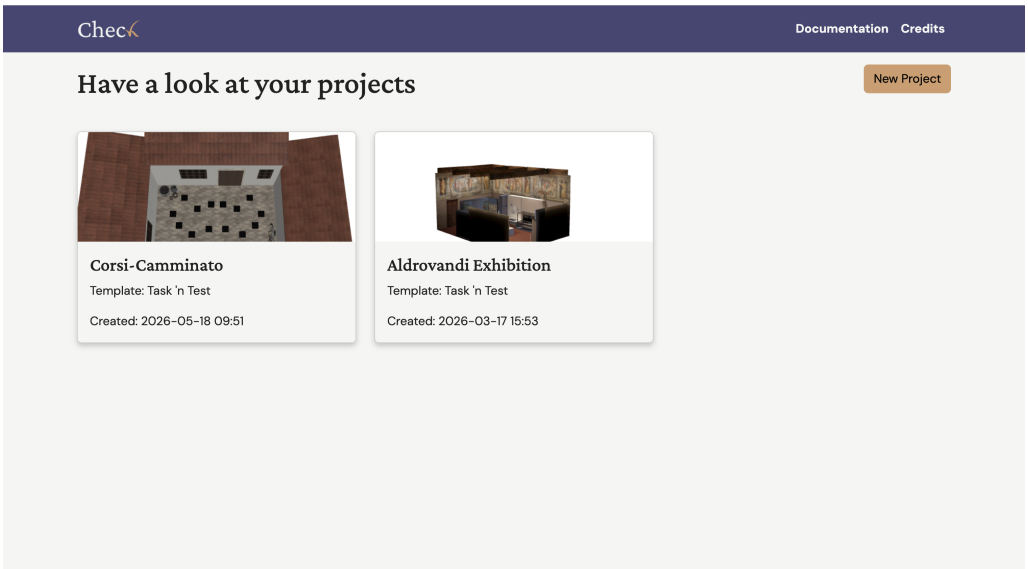


Figure 1: Main tester dashboard of CHECK, populated with two sample projects

4 Configuring a New Project

From the button "New Project", testers can create a new project. They are firstly prompted to choose between two pre-defined templates:

Free Wander : it assigns no explicit goal or navigation target to the participant. Once started, the participant is free to explore the virtual environment autonomously. The operator can observe and register movement patterns in real time through the live monitor panel and play different stimuli (images, text, audio recordings);

Task 'n Test : it contextualizes the exploration of the the subject in the completion of a specific task, described by an instruction accessible to the subject. Besides tracking immersive analytics and controlling stimuli and variables (as in the *Free Wander* template), the tester's dashboard keeps track of successful task completions, errors, or repetitions.

After this first selection, the tester provides further information on the project: the title and further information for documentation purposes on the goals, the sample, as well as the actions subjects will perform and how their behaviour will be measured and analysed throughout the experiment. Most importantly, the tester is asked to indicate: (a) the number of groups in which the sample should be subdivided; (b) if more than measure should be collected for each subject. This enables to refine variants of the protocol.

In the following touchpoint, testers upload the 3D environments in which the experiment will be set. A dedicated button enables to upload `*.glb` file; for each of them, a card with a thumbnail is generated (see Fig. 2). The interface also contains a button, which in future version of the application will allow to import scenes generated by H2IOSC Prototyper [9].

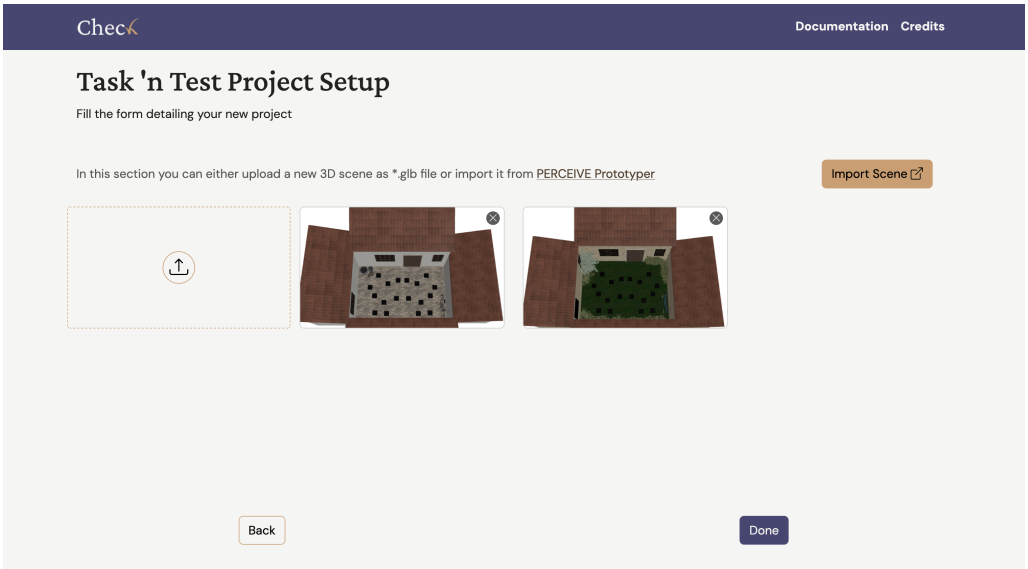


Figure 2: checK dashboard for the upload of 3D environments, here populated with two sample `*.glb` files

4.1 Protocol Definition

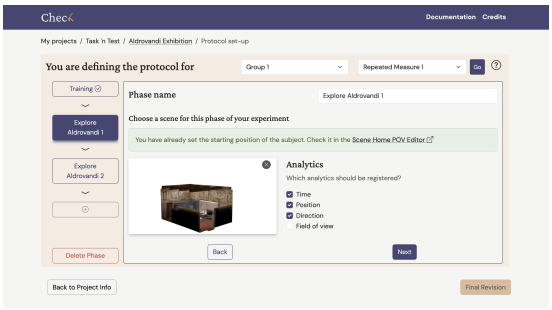
After an intermediate page summarising the data added in the previous steps ("First Project Summary" page), the key section of the authoring begins. It is handled via a single dashboard, which enables testers to configure the protocol variants for each group and measure, describing their single phases.

For each protocol, the system first asks if a training phase should be added: if the tester agrees, the first time the subject enters the immersive environment will follow a tutorial on how to explore the space. After the subject gets familiar with the controls, the experiment will continue as indicated by the protocol and its phases.

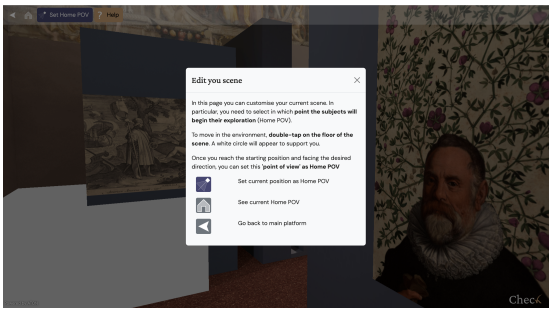
In this main authoring section, the dashboard is divided in two parts. On the left hand-side, the tester can edit the phases sequence, adding and deleting elements. On the right hand-side, parameters of the selected phases can be changed. All the necessary information are subdivided in three stacked forms, that the visitor can access in this order

1. The first touch point (Fig. 3a) requires to specify a custom name for the phase and, most importantly, the 3D environment in which the exploration / task will take place. The

tester can choose among the models loaded in the previous steps: the system will create an ATON scene starting from this model. After this choice, an alert will invite the tester to set the point where the starting position will begin (or "Home POV"), as shown in Fig. 3b. In this same page, the analytics to track during this phase will also be defined



(a) Definition of an ATON scene for the reference phase



(b) Instruction modal to edit the "Home POV" of an ATON scene

Figure 3: Functionalities of the first touch point of the phase configurator form

2. Exclusively for project of *Task 'n Test* template, two text area allow testers to specify the task the subject will perform during the phase. The first one, *description*, is aimed for documentation purposes; the second one is the proper *instruction* shared (orally or with a spatial interface) with the subject (see Sect. 5).
3. The last section pertains to the variables and multimedia content. Through a dedicated button, testers can upload images, video, audio tracks and texts. For each of them, they will be asked to distinguish if this multimedia stimulus is to be considered part of the instruction or will be shown later during the exploration as a variable. In this page, it is also possible to include as variable any animation embedded in the original `*.glb` file (see Fig. 4).

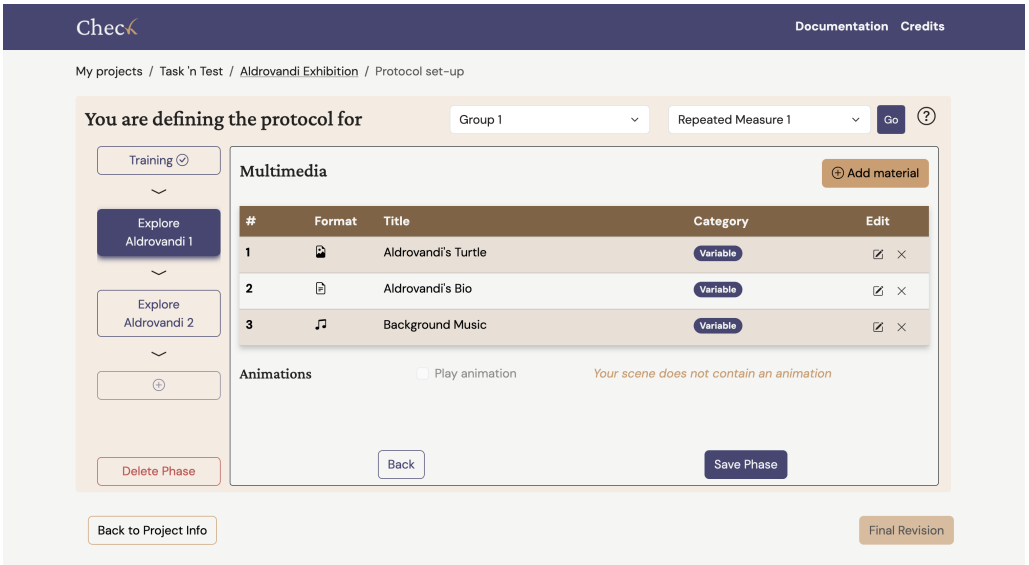


Figure 4: CHECK dashboard for the upload of multimedia assets as variables and instruction of a phases

The select menu in the top-right corner enables testers to access and edit protocols for different

groups and measures. Once a protocol variant is available for each group and repeated measure foreseen in the project, data can be save and testers can proceed with the final revision.

4.2 Project Overview Page

The final revision is available in the "Project overview" page, which is also accessible by clicking on the cards in the main dashboard.

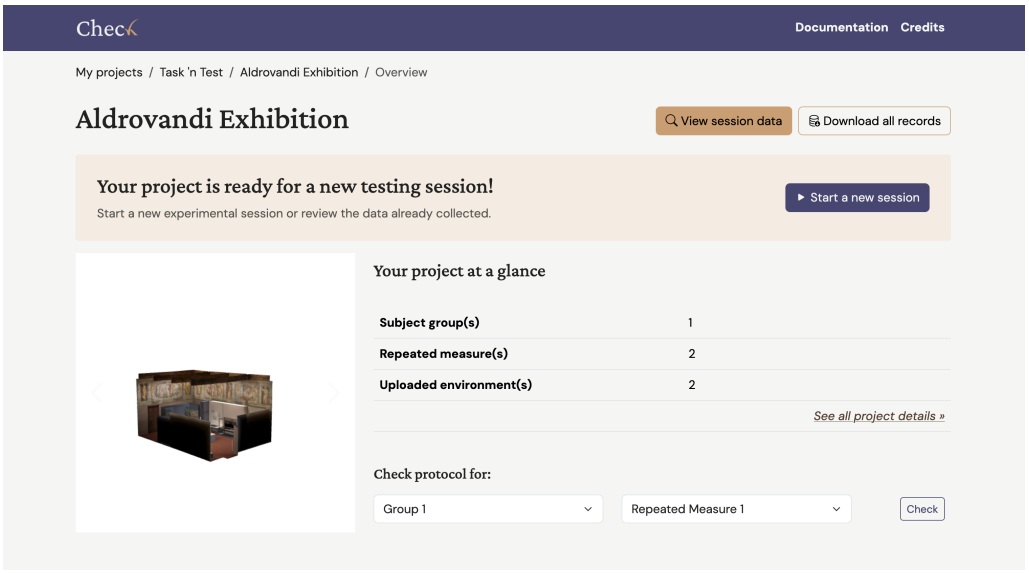


Figure 5: CHECK project overivew page handles the creation of new experimental sessions and see previously recorded data

If all protocol variants are defined and all other necessary pieces of information are provided (number of groups and repeated measure), the portal will appear as in Fig. 5). Otherwise, an alert will invite the tester to complete the missing informations. This page works as a main overview of the project and has three key function

1. Visualise main information of the project (environments, group, measure) and provide access to the "First summary" page, as well as protocol variant set-up pages, using the select menus at the bottom;
2. Start a new experimental session, by clicking on the button in the main alert (see Sect. 5);
3. Visualise data of a past experimental session, as well as exporting as `*.csv` file all the data recorded in the sessions of the project (see Sect. 6).

5 Running an Experimental Session

Clicking the alert on the "Project overview" page initialises a new experimental session. The tester is first presented with an onboarding form, where an anonymous ID code is assigned to the participant joining the session and the relevant protocol variant is declared by specifying the reference group and repeated measure. Confirming these values opens the main Session Dashboard (Fig. 6).

5.1 Subject's Interface

A dedicated onboarding page is also available to subjects at `http://localhost:\{port\}/a/checK/welcome/`. This page consists of a single input field requiring a *session ID* — a unique six-digit identifier communicated directly by the tester and visible in the tester's console. Once participants enter the immersive environment, they may begin their exploration.

During the session, the subject's interface is limited to the ATON scene and occasional modal overlays. A first modal welcomes the user and, if it was set in the protocol configuration phase, it provides instruction on how to use VR controllers. Secondly, in *Task 'n Test* template projects, participants first receive an instruction — either displayed as a spatial UI element or read aloud by the tester — prompting them to complete a specific task. These activities typically involve identifying an object or making a selection: subjects confirm their choice by pressing a designated key (e.g. `Enter` on desktop). This action triggers a response modal in the tester's dashboard, where the selection can be labelled as correct or incorrect.

A third class of modals is triggered by the *variables* activated by the tester. During the exploration, the tester may deliver multimedia assets to the subject — audio, visual, textual, or animated content — which are displayed as temporary overlays.

5.2 Tester's Interface

The tester's dashboard is divided into two columns. The left column displays contextual information about the ongoing session: at the top, the tester can read metadata about the project, the subject ID code, and the current *session ID*. This code must be communicated to the participant at the beginning of the session; an alert notifies the tester when the participant has successfully logged in.

The central element of this column is an embedded view of the scene, with the camera programmed to follow the subject's movements. The subject is represented by a coloured avatar.

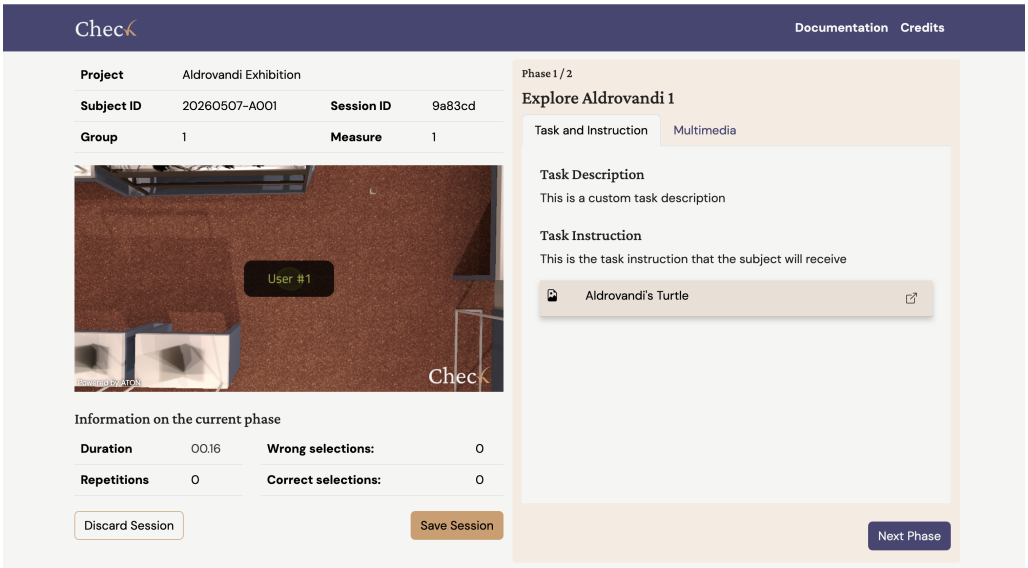
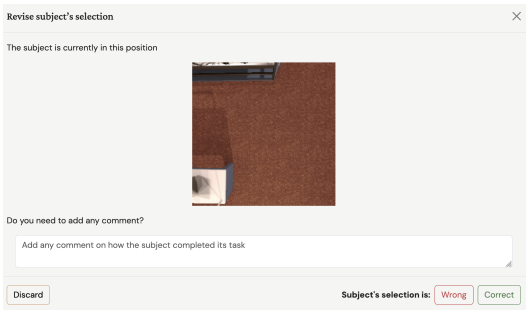


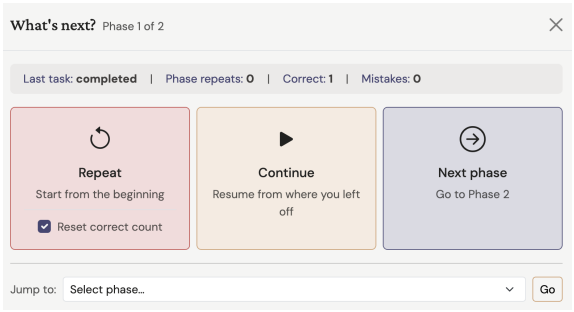
Figure 6: Main session dashboard of CHECK, with a sample *Task 'n Test* project.

The lower section of the left column displays performance statistics for the current phase. In *Free Wander* template projects, this consists exclusively of an elapsed-time counter. In *Task 'n Test* projects, three additional values are shown: the number of repetitions of the current phase, and the cumulative counts of correct and incorrect selections.

These values are updated in real time. When a subject confirms a selection, a first modal prompts the tester to label the response as correct or incorrect and to add any relevant remark (Fig. 7a). A second modal then allows the tester to decide how the session should proceed: (i) repeat the current phase, optionally resetting the selection count; (ii) continue the current phase; (iii) advance to the next phase, or end the session if the protocol is complete; (iv) jump directly to any phase in the protocol (Fig. 7b). While these modals are shown, the timer is paused and subject's exploration is temporarily blocked by an alert message.



(a) First modal: commenting subject's selection



(b) Second modal: selecting session next steps

Figure 7: Modals in tester's interface for *Task 'n Test* template projects.

The right column of the interface is dedicated to the variables of the current phase and is organised into two tabs. The first tab lists the task description and instructions, and is available exclusively in *Task 'n Test* template projects. The second tab presents all multimedia assets (*variables*) that the tester can trigger to deliver stimuli to the subject, as described in the previous section. Below these tabs, a button advances the session to the next protocol phase. Clicking "Save Session" instead terminates the experimental session and saves all collected data.

6 Records Visualisation and Data Export

From the Project Overview page, past experimental session records can be accessed by clicking "View Session Data". The visualisation displays data for one phase at a time. Upon loading, the tester must select the record to inspect through three linked dropdown menus: the first filters by participant, the second by session (each labelled with the corresponding repeated-measure index for easier retrieval), and the third by phase. Confirming these values updates the dashboard (Fig. 8).

In *Task 'n Test* template projects, the layout mirrors that of the session dashboard described in the previous section. The left column provides subject's metadata; its central element is an embedded ATON scene rendering the spatial trajectory followed by the participant as a blue polyline. This analytics feature is provided by the Interlumo service Merkhet [5].[3]

The right column presents aggregate performance statistics for the selected phase, together with a chronological timeline of the subject's selections: each entry records the selection made, the

[3] See the Interlumo platform at: https://interlumo.ispc.cnr.it/.

tester's assessment (correct or incorrect), any remarks added, and the next-step decision taken. This timeline is available exclusively in *Task 'n Test* projects. The *Free Wander* interface follows a simpler layout: the Merkhet web application occupies the full centre the page, providing a larger view of the trajectory data. In both templates, Merkhet web app can be visualised in full screen.

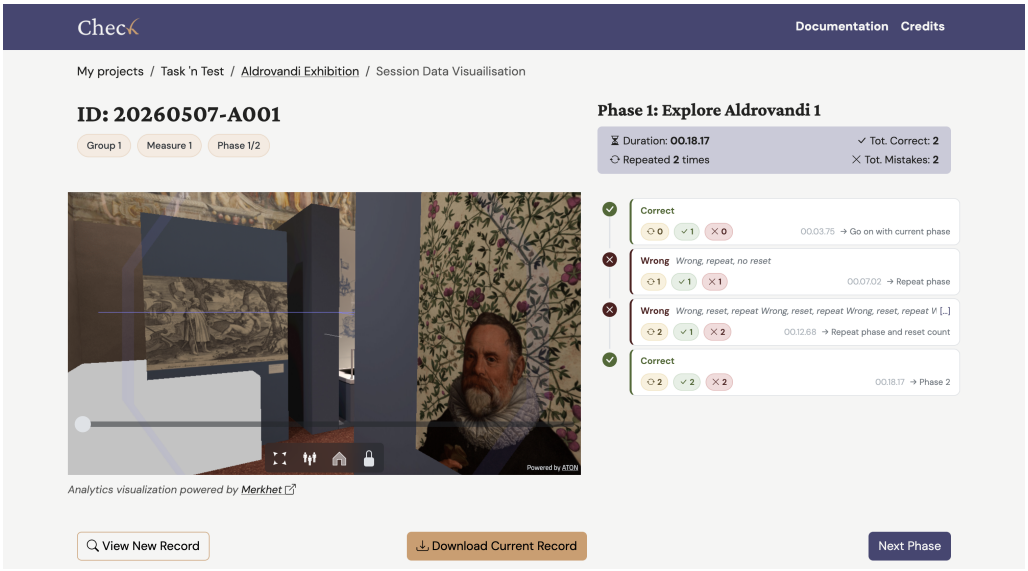


Figure 8: Session data visualisation in CHECK, with a sample *Task 'n Test* project.

At the bottom of the page, three buttons allow the tester to: (i) advance to the next phase of the current protocol; (ii) download the current record as a `*.csv` file; (iii) open a different record, returning to the three-menu selection form described above.

II. Technical Reference

7 Architecture and Codebase Structure

CHECK is designed as a web application, built with the ATON framework. As outlined in Sect. 2.2, it relies on two distinct modules:

1. a **flare**, acting as a server for the upload of the material (on the role of *flares* in ATON, please refer to [6]);
2. the main **web-app**, containing the pages, style sheets, scripts and other necessary material for the front-end of the application.

The server consists of two main assets: a client, in the folder `public/`, calling the flare on loading; and the main server (`check-server.js`), listing the principal routines for file uploads and directory creation.

The architecture of the web app codebase is instead richer and is described in the following listing:

```
checK/
+-- css/                    (style sheets)
+-- data/                   (project data storage)
+-- docs/                   (presentation website assets)
+-- img/                    (static images and icons)
+-- js/
|   +-- client-root.js      (root modules exposing common routines)
|   +-- project-data/       (scripts for data visualisation and export)
|   +-- project-run/        (scripts handling experimental session)
|   +-- project-setup/      (scripts involved in new project set-up)
+-- node_modules/           (Node.js dependencies)
+-- project-data/           (data visualisation HTML page)
+-- project-run/            (session management HTML pages)
+-- project-setup/          (project configuration HTML pages)
+-- welcome/
|   +-- index.html          (participant onboarding HTML page)
+-- index.html              (main dashboard HTML page)
```

The following sections will provide a more detailed description of these assets for each phase of the user journey

8 Style Sheet and Visual Identity

The web app is built using Bootstrap v5.3.0 (https://getbootstrap.com/), including the set of icons by Bootstrap Icon v1.13.1 (https://icons.getbootstrap.com/). Default tipography and colour palette have been both modified according to a new design system. This visual

identity is inspired by those of the two reference Horizon Europe projects for this platform: PERCEIVE (2023 - 2026) [11] and COLOURS (2025 - 2028) [3].

The colour palette have been chosen from the logos of the two project, and consists of a muted indigo (`#464673`) as primary colour and a warm sand (`#D19C6B`) as accent. The choice of these shades were also guided by Figma Colours (`https://www.figma.com/colors/`), where these shades are reported to be associated to wisdom and taking action. For background and body-colour respectively, an off-white (`#F5F5F4`) was preferred to pure white (`#FFFFFF`), whereas a warm black was (`#212120`) used instead of `#000000`. A contrast check was carried out before implementing the palette. As shown in Fig. 9, additional colours were added to respect original Bootstrap design system.



Figure 9: cheCK colour palette.

To what pertains to the typography, interfaces combined a serif font for headers, "Crimson Pro", and a sans-serif font for paragraphs and other body elements, "DM Sans". Both fonts are available on Google Font (`https://fonts.google.com/`). For better readability, default *root em* was increased to 112.5%, thus `1rem=18px`.



Figure 10: cheCK typography system.

Source Bootstrap stylesheet was customised via SASS (Syntactically Awesome Style Sheets,

https://getbootstrap.com/docs/5.1/customize/sass/). The result is available in `css/bootstrap-check.css`. Custom CSS declarations are instead asserted in `css/custom.css`.

9 Web Application Components

CHECK is a multi-page web application. Navigation between different sections of the same page is handled via URL parameters: `client-root.js` exposes a set of helper functions (e.g. the most important one, `getIdFromURL()`, retrieves the project from current URL) that parse the query string on each page load. State is never passed via JavaScript variables across pages, rather it is re-read from ATON App storage or from the URL itself.

9.1 Setup and Storage Architecture

CHECK relies on two parallel storage systems that operate concurrently and serve different purposes: the ATON App storage for structured JSON data, and the `check_server` filesystem for binary assets and exported records.

9.1.1 ATON App Storage

All JSON metadata is persisted through ATON's built-in key–value storage API (`ATON.App.getStorage` / `ATON.App.addToStorage` / `ATON.App.deleteFromStorage`). Storage keys follow a hierarchical convention rooted at `user-projects/`, defined in `client-root.js`:

(A) `user-projects/projects` : Global project index. Each entry contains minimal metadata (ID, label, creation timestamp) used to populate the main dashboard.

(B) `user-projects/{id}/config` : Full project configuration: template type, title, objectives, audience description, experimental actions, measure description, number of groups, and number of repeated measures.

(C) `user-projects/{id}/environments` : Index of uploaded 3D environments, keyed by asset ID. Each entry stores the GLB file descriptor, thumbnail URL, scene ID (once created), and animation flag.

(D) `user-projects/{id}/protocol/protocol-config` : Protocol definition, structured by a composite `"{measure},{group}"` key. Each key contains a `phase` object with sequential phase entries. Each phase stores name, ATON scene ID, analytics flags (`time`, `pos`, `dir`, `fov`), task description and instruction (*Task 'n Test* only), animation flag, and a map of assigned multimedia assets.

(E) `user-projects/{id}/protocol/asset/asset-library` : Project global asset library. Each entry records type (audio/image/video/text), file descriptors returned by the server, an optional custom display name, and creation timestamp.

9.1.2 Server-Side Filesystem

The `check_server` (Node.js / Express, port 3001, defined in `check-flare/check_server.js`) handles all binary file operations. It exposes a REST API and manages two filesystem sub-

trees: `wapps/checK/data/` (see listing below), and the two `check-user/` subfolders in `data/` directory.

`wapps/checK/data/user-projects/{id}/` : Project metadata folder. Created at project initialisation (`POST /projects`), it contains JSON storage files (see *supra*), and the subfolder for uploaded multimedia files (audio, image, video), i.e. `protocol/asset/`. This tree is statically served at `/user-projects/` to allow direct URL access to assets during sessions.

`wapps/checK/data/session/` : Session records (`records.json`, F in the listing below) and exported CSV files, written by `POST /session/:projectId/:sessionCode/csv`.

`data/collections/check-user/models/{id}/` : GLB models and PNG thumbnails, stored inside the main Aton collections tree so they are served directly by the Aton static file server at `http://localhost:{port}/collections/. . .`

`data/scenes/check-user/{sceneId}/scene.json` : Aton scene descriptors, written by `PUT /projects/:projectId/upload/scenes/:sceneId` when a 3D environment is first associated with a protocol phase.

```
checK/
+-- data/
|   +-- session/
|   +----- records.js                              (F: past sessions index)
|   +----- <sample-session>.csv
|   +-- user-projects/
|   +----- projects.js                             (A: global project index)
|   +----- <sample-project>/
|   +-------- config.json                          (B: full project configuration)
|   +-------- environments.json                    (C: uploaded environments index)
|   +----------- protocol/
|   +-------------- protocol-config.json           (D: protocol configurator)
|   +-------------- asset/
|   +----------------- asset-library.json          (E: uploaded assets index)
|   +----------------- <uploaded-asset-file>
```

Note: JSON metadata is never written by the server. The server only handles binary uploads, file deletions, and CSV export. All JSON state is managed exclusively from the client via Aton App storage.

9.2 New Project Creation

The creation of a new project spans three distinct pages and involves four JavaScript modules: `client-setup.js`, `pagination-project.js`, `pagination-protocol.js`, and `3D-thumb.js`.

9.2.1 Project Initialisation

A new project is created from `index.html` by clicking the "New Project" button (see Fig. 1), which calls `createProjectFromUI()` in `client-setup.js`. The function generates a timestamp-based identifier in the format `YYMMDDHHmm` (e.g. `2503141045`) and `POST`s it to `/projects` on the check server. The server creates the full folder structure (see Section 9.1) and returns the new project metadata. The client then redirects to `project-setup/project-form.html?id={projectId}&n=0`.

9.2.2 Project Form Wizard

The project form (`project-form.html`) is a single HTML page rendered as a multi-section wizard. Navigation is managed entirely by `pagination-project.js`, which reads the current section index from the URL parameter `n` via `getSectionIndexFromURL()` and shows only the corresponding `.new-project-form-section` div, hiding all others. Moving forward or backward calls `goToFormSections()`, which: saves the current section's data to `localStorage` via `saveFormLocally()`; flushes it to ATON App storage via `saveProjectConfigFromForm()` (at steps 1 and 2 only); then reloads the page with the incremented `n` parameter.

The wizard comprises four sections:

Section 0 — Template selection : The tester chooses between *Task 'n Test* and *Free Wander*. The choice is persisted to `localStorage` under the key `isTaskTest` via `setIsTaskTest()`, so it survives page reloads and is readable across the rest of the setup flow.

Section 1 — Project description / 1st Part : Title, objectives, and audience. Field values are buffered in `localStorage` on navigation and flushed to ATON storage under the `config.json` key on confirmation.

Section 2 — Project description / 2nd Part : Actions subjects will perform during the experiment, measure description, number of groups, and number of repeated measures. Written to the `config.json` storage key on confirmation.

Section 3 — 3D environment upload : The tester uploads one or more GLB or GLTF files (Fig. 2). Each upload triggers `upload3DAssetForCurrentProject()` in `client-setup.js`, which first calls `generateGlbThumbnail()` (`3D-thumb.js`) to render an offline preview, then `POST`s both the GLB and the thumbnail PNG to `/projects/:id/upload` on the server. Finally, it writes the returned asset descriptor to the `environments` storage key.

On completion, the wizard redirects to `project-setup/project-summary.html`.

9.2.3 GLB Thumbnail Generation

Thumbnail generation is handled by an Immediately Invoked Function Expression (IIFE) module in `3D-thumb.js` that runs entirely in the browser. On initialisation, it creates an off-screen `THREE.WebGLRenderer` (512×512 px), positioned at `left: -9999px` to be invisible to the user, with a white background and two directional lights plus a hemisphere light.

When `generateGlbThumbnail(file)` is called, the module relies on `THREE.GLTFLoader` to load the GLB, auto-frames the model using a bounding-box fit (`frameObject()`), renders a single frame, and extracts the result as a PNG blob via `canvas.toBlob()`.

The function also detects the presence of animations (`gltf.animations.length > 0`) and returns a `hasAnimation` flag alongside the blob, which is stored in the asset descriptor to conditionally enable the animation toggle in the protocol setup.

9.2.4 Protocol Setup

After the project summary, the tester proceeds to protocol configuration (`protocol-setup.html`). This is handled by the module `pagination-protocol.js`: it reads five URL parameters on every page load: `p` (phase index), `s` (step index), `g` (group), `m` (repeated measure), and `t` (template:

0 = *Free Wander*, 1 = *Task 'n Test*). Page-to-page navigation is handled by `updatePage()`, which merges a partial parameter object into the current query string and reloads the page — state is thus always encoded in the URL, never in JavaScript variables across reloads.

Each protocol is keyed by the composite string `"{measure},{group}"` in the ATON storage. A first call to `initialiseProtocolPhases()` seeds the key with a training phase (phase 0) and a default first experimental phase (phase 1). The phase panel on the left is rendered dynamically by `uploadContentInPhase()`, which reads all existing phases from storage and creates one Bootstrap button per phase; the active phase button is highlighted by swapping `btn-outline-primary` for `btn-primary`.

Each phase is edited through up to three steps, whose availability depends on the template:

Step 1 — 3D environment and analytics : The tester selects a 3D environment from the project's asset library (displayed in a modal). On confirmation, `setPhaseEnvironment()` calls `realizeATONSceneConfig()`. If the environment has not been used in any previous phase, it creates new `scene.json` on the server through the REST API method `PUT /upload/scenes/:sceneId`. The scene ID is then persisted on the environment entry in storage. Analytics flags (position, direction, field of view, time) are toggled via checkboxes and saved with the phase.

Step 2 — Task description and instruction : Available in *Task 'n Test* only. The tester writes the task description and the instruction displayed to the subject.

Step 3 — Multimedia assets : The tester builds the asset list for the phase from the project's asset library. Each entry can be typed as an *Instruction* (shown before the task) or a *Variable* (triggered on demand during the session). Assets are managed through an inline table; rows can be added, edited, and deleted without leaving the page. The animation toggle is enabled only if the selected 3D environment carries a skeletal animation (`hasAnimation` flag) (see Fig. 4).

> **Note: Scene Editor.** From Step 1 of each phase, the tester can open `protocol-scene-editor.html` to set the participant's starting point of view ("Home POV") within the 3D environment. Relying on ATON front-end HATHOR, the page loads an ATON and enables to write a "Home POV" (position, target, and field of view) directly to the scene JSON on the check server (`PUT /upload/scenes/:sid`). The protocol setup page checks whether a Home POV has been defined and displays a warning alert until one is set.

Phase deletion (`deletePhaseFromConfig()`) re-numbers all subsequent phases to maintain a contiguous index; phase 1 cannot be deleted but is reset to its default state instead. A new phase is unlocked and added only after the current one is saved (`saveAndUnlockPhase()`).

9.3 Experimental Session: Architecture and Runtime

The experimental session is distributed across three HTML pages and five JavaScript modules under `project-run/`. The responsibilities are clearly separated: `client-exprun.js` handles storage access and session state; `dashboard-manager.js` owns the tester dashboard UI; `session-manager.js` manages phase navigation and the modal flow; `data-manager.js` handles data capture and CSV export; `aton-main.js` manages the ATON scene, the Photon multi-user layer, and the subject-facing interface. Lastly, `play-media.js` handles the delivery of multimedia assets to the subject.

9.3.1 Session Initialisation

The tester starts a session from the Project overview page: as described in Sect. 5, it redirects to an onboarding form (`onboarding.html`) where the tester assigns an anonymous pseudocode to the subject and provides additional information. Later, `createSessionRecord()` (`data-manager.js`) generates a random six-character hexadecimal session code via the Web Crypto API and writes a new entry to ATON App storage `records.json`. The tester is then redirected to `experiment.html?id={projectId}&sc={sessionCode}`.

9.3.2 Storage Snapshot

In every page load within session flow, `updateStorageObjects()` (`client-exprun.js`) reads the full session state from ATON App storage into five shared module-level variables:

`sessionRecord` : Subject ID, group, measure, and project ID, resolved from `session/records` by matching the session code from the URL parameter `sc` (referenced with F in the last code listing).

`projectConfig` : Full project configuration including template type and title.

`protocolConfigStorage` : Full protocol definition, keyed by `"{group},{measure}"`.

`phasesObj` : The phase map for the current group/measure combination, extracted from `protocolConfigStorage`.

`protocolMultimediaLibraryStorage` : The full asset library for the project.

9.3.3 Tester Dashboard

`experiment.html` is the main tester interface. The ATON 3D scene is loaded inside an `<iframe>` (`#testerATONSceneFrame`) pointing to `scene.html`. This iframe architecture allows the tester page and the ATON scene to run as separate JavaScript contexts that communicate via the calls `window.parent` and `iframe.contentWindow`.

On load, `dashboard-manager.js` calls `updateStorageObjects()`, populates the session metadata header, and calls `updatePhase(0)` to initialise the first phase. This routine coordinates two operations:

- `loadPhaseATONScene()` updates the `iframe src` to load the correct scene for the phase (using phase 1's scene also for the training phase);
- `updateRightPanel()` populates the right column according to template and phase. For *Free Wander*, the Task tab is disabled and the Multimedia tab is activated by default. For *Task 'n Test*, both tabs are available.

The Task tab shows the task description, instruction, and instruction-type assets. The Multimedia tab shows variable-type assets and, if configured, the 3D animation toggle. Asset tables are populated by `addMultimediaTable()`, which reads `protocolMultimediaLibraryStorage` and `phasesObj` to render one row per asset with a preview button (`previewAsset()`) and, for variable-type assets, a delivery button. The delivery button calls `playMedia(assetName)` (see *infra*), which resolves the asset and delivers it to the subject's interface: as a spatial VR panel if

an active WebXR session is running, or as a modal overlay in the standard browser. A **Stop** button calls `window.stopPlayMedia()` to dismiss the panel or audio.

9.3.4 Multimedia Delivery

Multimedia asset delivery is handled by the self-contained module `play-media.js`, loaded in `scene.html` after `client-exprun.js`. The module exposes `window.playMedia(assetName)` and `window.stopPlayMedia()` as global functions:

`playMedia()` resolves the asset from `protocolMultimediaLibraryStorage` by matching either the asset storage key or its `customName` field. It then dispatches the asset according to the current rendering context:

WebXR session (VR) : A spatial Three.js `Group` is built and attached to the Aton scene root. The panel follows the user's position via `requestAnimationFrame`, a XR-session-aware continuous animation loop, positioned `FORWARD_FACTOR` metres ahead, `LEFT_FACTOR` metres to the left, and `UP_FACTOR` metres above the user's eye level, always facing the user. Text planes are rendered via Canvas2D onto a `THREE.CanvasTexture`; image planes use `THREE.TextureLoader`; video planes use `THREE.VideoTexture`.

Standard browser (Web) : The asset is displayed in an `ATON.UI.showModal` overlay (if available) or in a Bootstrap 5 modal injected dynamically into `document.body`. The modal title is set to the asset's `customName`.

Audio only : In both contexts, audio-only assets play via a reused `<audio>` element without any visual feedback.

The supported asset type combinations are: *text*, *image*, *video*, *audio*, *text+image*, *text+video*, and any of the above combined with *audio*. When text and a visual medium are combined, the text plane is stacked above the media plane with a configurable gap (`PANEL_GAP`).

`stopPlayMedia()` pauses and clears the audio element and removes the active VR panel, disposing all Three.js geometries, materials, and textures to avoid memory leaks.

9.3.5 Scene and Multi-User Layer

The subject interface consists of a first onboarding form, requiring a *session ID*. The function `enterSession()` decodes this hex-code in a series of query parameters string and redirects to `scene.html`.

This page is a minimal Aton shell and is used both as the iframe inside the tester dashboard and as the full-screen environment for the participant (or immersive environment for VR navigation). Its behaviour is entirely determined by the URL parameter `r`: `r=0` identifies the tester; `r=1` the subject.

This page is fully handled by `aton-main.js`: this module initialises Aton, loads the scene identified by `sid`, and connects to Photon for multi-user synchronisation. Role diverges immediately after connection:

Tester (`r=0`) : Orbit navigation is activated. `bSendState = false` prevents the tester's avatar state to be broadcast: the tester can hence observe the subject's avatar without being visible in the environments. The tester listens for Photon events fired by the participant.

Subject (r=1) : First-person navigation is activated. On connection, the subject fires the event `triggerAlert` to notify the tester of their presence; the tester receives a SweetAlert2 (https://sweetalert2.github.io/) dialog and starts the phase timer on confirmation. The subject listens for scene-load, home-POV, animation, and session-close events from the tester.

On connection, the subject's interface shows a welcome modal before any tester interaction. The content of the modal depends on whether a training phase (phase 0) exists in the protocol: if it does, the modal includes navigation instructions for teleport-based movement; otherwise it prompts the subject to wait for the tester's instructions. This check is performed by `aton-main.js` via `updateStorageObjects()`, reading the `phasesObj[0]` flag.

The key Photon events and their direction are:

`triggerAlert` (subject → tester) : Fired by the subject immediately on Photon connection, carrying the subject's session code. The tester receives a SweetAlert2 confirmation dialog; on confirmation, the phase timer is started via `startTimer()`. Note that the timer state is checked before starting: if the timer is already running or has a non-zero value, `initTimer()` is called first to reset it.

`updateTesterPos` (subject → tester) : Sent at high frequency via `setInterval`; carries a computed overhead POV (participant position $+1.75$ m on the y-axis) so the tester's camera follows the participant from above.

`showModal` (subject → tester) : Fired when the participant presses `Enter` (or the VR trigger). Carries the participant's current POV; `aton-main.js` uses it to take a screenshot via `ATON.Utils.takeScreenshotFromPOV()` and passes it to `subjectTrigger()` in `session-manager.js`, opening the first modal in the tester dashboard (Fig. 7a).

`loadScene` (tester → subject) : Redirects the participant to a new scene URL, appending Kapto analytics parameters (`mk.hub`, `mk.freq`, `mk.attr`, `mk.dur`, `mk.gid`).

`homePOV` (tester → subject) : Sends the participant back to the scene's Home POV.

`showIntroModal` (tester → subject) : Sent at session start, it delivers a boolean `hasIntro` that controls which welcome message is shown to the subject in the immersive environment: if `true`, navigation instructions are included; if `false`, the subject is instructed to wait for the tester.

`requestPlayAnimation` (tester → participant) : Synchronises the 3D animation playback. On the tester side, `toggle3DAnimation()` fires the event and listens for `animationFinished`: this custom event is dispatched by the iframe when the Three.js animation mixer finishes. Its inverse is `requestStopAnimation`.

`closeSession` (tester → subject) : Shows a closing modal to the participant when the session ends.

9.3.6 Phase Navigation and Data Capture

When the subject makes a selection in a *Task 'n Test* project, a first modal opens in the tester dashboard (`#modalSelectionViewer`, see Fig. 7a). The tester labels the selection as correct, incorrect, or discarded and adds an optional comment. `changePhaseModal()` updates the live counters (`#phase-correct`, `#phase-wrong`) and calls `updateResultsDataChunk()` to write the outcome into the active data chunk. A second modal (`#modalPhaseChange`, Fig. 7b) then

presents the tester with four options: repeat the phase (optionally resetting the correct count), continue the current phase, advance to the next phase, or jump to any phase. These behaviours are handled by `handlePhaseAction()`.

The capture of these data chunks is handled by the module `data-manager.js` and follows a two-step pattern. `startCaptureDataChunk()` initialises a new `dataChunk` object with a timestamp, the current phase index, and the Merkhet ID (see *infra*). Two subsequent calls to `updateResultsDataChunk()` add the outcome fields and the next-action decision respectively; only the second call appends the completed chunk to `sessionData`. At session end, this array is serialised as a CSV string by `sessionDataToCSV()`, and `saveSessionCSV()` POSTs it to `/session/:projectId/:sessionCode/csv` on the check server. The file appears in the folder `checK/data/session/`.

The reference to the Merkhet session ID (`mkid`) in each data chunk connects the interaction logs recorded by checK with the spatial analytics captured by the Interlumo Kapto service for the same phase. The configuration of the captured analytics is set directly in the query parameters of the subject's interface and are asserted by the function `enterSession()`. The call to Merkhet and Kapto flares takes place in `client-root.js`.[4] The registration of immersive analytics does not differ between the two templates.

9.4 Data Visualisation and Bulk Export

Session record visualisation and data export are accessible on two ad hoc buttons in the Project Overview Page (see Fig. 5).

9.4.1 Session Data Visualisation

The data visualisation page is accessible from the button "View Session Data" and is served at `project-data/index.html?id={projectId}`. It is driven by `data-viz.js` (see Section 6 for the user-facing description).

On load, `updateStorage()` reads the project configuration, the protocol definition, and the full session index (`session/records`) into module-level variables. The page presents three cascading `<select>` (subject, session, phase) menus, each enabled only after the previous one has been confirmed.

On confirmation, `displayCurrentData()` fetches the session CSV from the Aton static server, parses it into an array of row objects , and filters the rows to the selected phase via `row.currentPhase`. The dashboard is then updated by two functions:

- `updateMerkhetFrame()` constructs the Merkhet viewer URL from the phase's `sceneID` and the `mkid` field of the first data row (see *supra*), loads it inside the `#MerkhetContainer` iframe.
- `updateUI()` populates the left column with subject ID, group, measure, phase label, phase duration (taken from the `timeStamp` of the last row), and a direct download link to the session CSV.

This latter function distinguishes between:

[4] Read Kapto documentation at: https://interlumo.ispc.cnr.it/kapto/api-docs-client/Kapto.html.

Task 'n Test : It updates the count of correct and incorrect selection, and repetition iterating on the rows of the selected dataset. On the right handside, `renderTimeline()` iterates `currentData` and renders one card per row, showing the outcome (correct/wrong), any tester comment, running counters for repetitions and correct and wrong selections, the row timestamp, and the next-action decision.

Free Wander : Selection stats and the timeline are hidden. The Merkhet iframe is expanded to full page width and centred, providing a larger view of the spatial trajectory.

9.4.2 Bulk CSV Export

The button "Download all records" is placed in the project overview page and calls the omonymous function in `bulk-download.js`. On page load, the module reads the full session index from ATON App storage (`session/records/{projectId}`), then fetches and parses the individual CSV file for each session from `/a/checK/data/{csvPath}` on the ATON static server. Each row is enriched with four session-level fields: `sessionID`, `subjectID`, `group`, `measure`. These values are appended to a shared `projectData` array.

When the button is clicked, `writesCSVfromArray()` serialises `projectData` into a single CSV file. Column order is normalised: the five priority fields (`sessionID`, `subjectID`, `group`, `measure`, `timeStamp`) always appear first, followed by the remaining keys. As for the download of data of the single record, the output file is downloaded directly in the browser via a temporary object URL, without any server-side involvement.

10 Future Work

Future development directions include the introduction of user authentication and multi-user collaboration features, which would allow research teams to share projects, assign roles, and manage access to experimental data across research units. A second priority is the integration with the H2IOSC Prototyper [9], which would enable CHECK to operate within a broader open-science infrastructure for digital cultural heritage, guiding practitioners from the creation of 3D scenes to their reuse in experimental contexts.

Credits

The application was developed by CNR ISPC - Digital Heritage Innovation Lab (Director: Prof Pescarin) and was funded by: the project H2IOSC (https://www.h2iosc.cnr.it/, funded by the European Union NextGenerationEU – CUP B63C22000730005); the CHANGES foundation (https://sites.google.com/uniroma1.it/changes, funded by the European Union NextGenerationEU - CUP B53C22003780006); the Horizon Europe projects PERCEIVE (https://perceive-horizon.eu/, funded by the European Union under grant agreement 101061157) and COLOURS (https://colours-eccch.eu/, funded by the European Union under grant agreement 101233413).

The project benefited of the collaboration with EPFL Laboratory for Experimental Museology, with funding of the Swiss Government Excellence Scholarship No. 2025.0089. We also thank Prof. Nori (University of Bologna), Prof. Piccardi and Dr. Von Gal (University of Rome "La Sapienza") for their support in this study.

References

[1] Brainard, D. H. The Psychophysics Toolbox. *Spatial Vision 10*, 4 (1997), 433–436.

[2] Brookes, J., Warburton, M., Alghadier, M., Mon-Williams, M., and Mushtaq, F. Studying human behavior with virtual reality: The Unity Experiment Framework. *Behavior Research Methods 52*, 2 (Apr. 2020), 455–463.

[3] COLOURS. Collaborative On-cloud Lab for the conservation and digital restoration of ColOUred heritage collectionS, 2025. European Commission, CORDIS.

[4] Fanini, B., Ferdani, D., Demetrescu, E., Berto, S., and d'Annibale, E. ATON: An Open-Source Framework for Creating Immersive, Collaborative and Liquid Web-Apps for Cultural Heritage. *Applied Sciences 11*, 22 (Nov. 2021), 11062.

[5] Fanini, B., and Gosti, G. A New Generation of Collaborative Immersive Analytics on the Web: Open-Source Services to Capture, Process and Inspect Users' Sessions in 3D Environments. *Future Internet 16*, 5 (Apr. 2024), 147.

[6] Fanini, B., Gosti, G., Amadei, C. F., and Menconero, S. Shaping a Web3D framework for different scientific communities: ATON as a service in H2IOSC. In *Digital Heritage 2025* (2025), S. Campana, D. Ferdani, H. Graf, G. Guidi, Z. Hegarty, S. Pescarin, and F. Remondino, Eds., The Eurographics Association.

[7] Kleiner, M., Brainard, D., Pelli, D., Ingling, A., Murray, R., and Broussard, C. What's new in psychtoolbox-3. *Perception 36* (Jan. 2007), 1–16.

[8] Massidda, M., Bordignon, A., Fabbri, F., Nori, R., Piccardi, L., Travaglini, L., Veggi, M., and Pescarin, S. Towards an Experiment Planner for Cognitive Studies in Virtual Heritage Environments. A Pilot Study. In *Eurographics Workshop on Graphics and Cultural Heritage* (2024), M. Corsini, D. Ferdani, A. Kuijper, and H. Kutlu, Eds., The Eurographics Association.

[9] Massidda, M., Fanini, B., and Pescarin, S. Prototyper: A web3d platform for collaborative design and simulation of hybrid museum exhibition. In *Digital Heritage 2025* (2025), S. Campana, D. Ferdani, H. Graf, G. Guidi, Z. Hegarty, S. Pescarin, and F. Remondino, Eds., The Eurographics Association.

[10] Peirce, J. W. PsychoPy—Psychophysics software in Python. *Journal of Neuroscience Methods 162*, 1-2 (2007), 8–13.

[11] PERCEIVE. Perceptive Enhanced Realities of Colored collEctions through AI and Virtual Experiences, 2023. European Commission, CORDIS.

[12] WorldViz. SightLab VR Pro: Vr experiment generator and eye tracking software, 2024.